



BLM4522 Ağ Tabanlı Paralel Dağıtım Sistemleri

Veritabanı Performans Optimizasyonu ve İzleme,

GitHub: <https://github.com/ismailbsrn/DatabaseScenarios-3>

Video:

İsmail Başaran

22290137

Mayıs 2026

1. Veritabanı Performans Optimizasyonu ve İzleme

Raporun birinci bölümünde veritabanı performans optimizasyonu ve izleme senaryosu ele alınmaktadır.

1.1. Projenin Amacı

Bu projenin temel amacı, yapay şekilde büyük bir veri tabanı oluşturmak ve bu veri tabanı üzerinde gerçekleştirilen sorguların performansının izlenmesi ve optimize edilmesidir. Veri tabanı yönetim sistemi olarak PostgreSQL kullanılacaktır.

1.2. Proje Kapsamı

Proje aşağıdaki özellikleri içermektedir:

- PostgreSQL için pg_stat_statements eklentisi ile performans takibi.
- Sorgu performansı için ihtiyaç duyulan yerlerde indeks kullanımı.
- Uzun süren sorguları inceleme ve optimize etme.
- Farklı roller için erişim yönetimi.

1.3. Gerçekleştirim ve Yöntem

Bahsedilen aşamaların nasıl gerçekleştirildiği detaylarıyla birlikte bu bölümde açıklanmaktadır.

1.3.1. Veritabanı İzleme

Bu adımda yapay veriler ile büyük bir veri tabanı hazırlanmış ve bu veri tabanı üzerinde gerçekleştirilen sorguların performansı izlenmiştir.

```
1 CREATE TABLE transactions (  
2   id SERIAL PRIMARY KEY,  
3   customer_id INT,  
4   transaction_date TIMESTAMP,  
5   amount NUMERIC(10, 2),  
6   status VARCHAR(20)  
7 );  
8 INSERT INTO transactions (customer_id, transaction_date, amount, status)  
9 SELECT  
10  floor(random() * 100000 + 1)::int,  
11  timestamp '2023-01-01 00:00:00' + random() * (timestamp '2026-01-01 00:00:00' - timestamp '2023-01-01 00:00:00'),  
12  (random() * 10000)::numeric(10, 2),  
13  (ARRAY['success', 'failed', 'pending', 'refunded'])[floor(random() * 4 + 1)]  
14 FROM generate_series(1, 10000000);
```

Yukarıdaki sql scripti ile 10 Milyon satırlık bir tablo oluşturulmuştur.

```
DatabaseScenarios-3 $ psql -U ismailbasaran -d ornek_veritabani < create_db.sql
CREATE TABLE
INSERT 0 10000000
DatabaseScenarios-3 $ psql -U ismailbasaran -d ornek_veritabani
psql (18.3 (Homebrew))
Type "help" for help.

ornek_veritabani=# SELECT pg_size_pretty(pg_relation_size('transactions')) AS disk_boyutu, count(*) AS toplam_satir
FROM transactions;
 disk_boyutu | toplam_satir 
-----+-----
 574 MB      | 100000000
(1 row)

ornek_veritabani=#
```

Yukarıdaki görüntüde görüleceği üzere transactions tablosu 10 Milyon satırdan oluşmaktadır ve diskte 574 Mb yer kaplamaktadır. Performans izleme işlemi için pg_stat_statements eklentisinin kurulması gerekmektedir.

```
ornek_veritabani=# CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
CREATE EXTENSION
```

Yukarıdaki komut ile pg_stat_statements eklentisi kurulmuştur. Bu noktadan sonra veri tabanı üzerinde çalıştırılan sorguların performansı izleme altındadır.

```
ornek_veritabani=# EXPLAIN ANALYZE
SELECT count(*)
FROM transactions
WHERE status = 'failed' AND amount > 9499;

QUERY PLAN
-----
Finalize Aggregate  (cost=137159.31..137159.32 rows=1 width=8) (actual time=209.540..210.523 rows=1.00 loops=1)
  Buffers: shared hit=12111 read=61419
    -> Gather  (cost=137159.10..137159.31 rows=2 width=8) (actual time=209.488..210.518 rows=3.00 loops=1)
          Workers Planned: 2
          Workers Launched: 2
          Buffers: shared hit=12111 read=61419
            -> Partial Aggregate  (cost=136159.10..136159.11 rows=1 width=8) (actual time=203.786..203.786 rows=1.00 loops=3)
                  Buffers: shared hit=12111 read=61419
                    -> Parallel Seq Scan on transactions  (cost=0.00..136030.50 rows=51438 width=0) (actual time=0.213..202.312 rows=41814.33 loops=3)
                          Filter: ((amount > '9499'::numeric) AND ((status)::text = 'failed'::text))
                          Rows Removed by Filter: 3291519
                          Buffers: shared hit=12111 read=61419
Planning Time: 0.154 ms
Execution Time: 210.607 ms
(14 rows)
```

Sonraki aşamalarda görmek üzere uzun zaman alacak yukarıdaki rapor sorgusu çalıştırılmıştır.

```
ornek_veritabani=# SELECT query, calls, round(total_exec_time::numeric, 2) AS total_time_ms, rows
FROM pg_stat_statements
ORDER BY total_exec_time DESC LIMIT 5;
 query | calls | total_time_ms | rows 
-----+-----+-----+-----
 EXPLAIN ANALYZE | 1 | 210.81 | 0
 SELECT count(*) |  |  | 
 FROM transactions |  |  | 
 WHERE status = $1 AND amount > $2 |  |  | 
 SELECT pg_stat_statements_reset() | 1 | 0.50 | 1
(2 rows)
```

Ardından yukarıdaki sorgu ile veri tabanı üzerinde gerçekleştirilen ve en çok zaman alan 5 sorgu ekrana yazdırılmıştır. Görüleceği çalıştırılan rapor sorgusu 210ms sürmektedir. Aktif olmayan örnek bir veri tabanı üzerinde çalışıldığından detaylı bir analiz yapılamıyor olsa da pg_stat_statements aracı ile bu izleme işleminin gerçekleştirilebileceği gösterilmiştir.

1.3.2. Index Yönetimi

Raporun bu bölümünde bir önceki aşamada gerçekleştirilen rapor sorgusunu hızlandırmak için tablodaki status ve amount sütunlarına Composite Index eklenecektir.

```
ornek_veritabani=# CREATE INDEX idx_transactions_status_amount ON transactions(status, amount);
CREATE INDEX
ornek_veritabani=#
```

Bu noktada performansı optimize etmek için yukarıdaki sorgu ile status ve amount sütunlarına Composite Indexleme işlemi yapılmıştır.

```
ornek_veritabani=# SELECT pg_stat_statements_reset();
pg_stat_statements_reset
-----
2026-05-06 11:23:42.760895+03
(1 row)

ornek_veritabani=# EXPLAIN ANALYZE
SELECT count(*)
FROM transactions
WHERE status = 'failed' AND amount > 9499;

                                QUERY PLAN
-----
Aggregate  (cost=4682.16..4682.17 rows=1 width=8) (actual time=35.478..35.479 rows=1.00 loops=1)
  Buffers: shared hit=67717 read=1
  -> Index Only Scan using idx_transactions_status_amount on transactions  (cost=0.56..4373.54 rows=123449 width=0) (actual time=0.153..26.419 rows=125443.00 loops=1)
    Index Cond: ((status = 'failed'::text) AND (amount > '9499'::numeric))
    Heap Fetches: 0
    Index Searches: 1
    Buffers: shared hit=67717 read=1
Planning Time: 0.177 ms
Execution Time: 35.515 ms
(9 rows)
```

Indexleme işlemi sonrasında aynı sorgu tekrar çalıştırılacağından ne kadar sürdüklerini ayrı ayrı görebilmek için pg_stat_statements eklentisinin hafızası resetlenmiş ve sonrasında aynı sorgu tekrar gerçekleştirilmiştir.

```
ornek_veritabani=# SELECT query, calls, round(total_exec_time::numeric, 2) AS total_time_ms, rows
FROM pg_stat_statements
ORDER BY total_exec_time DESC LIMIT 5;

      query      | calls | total_time_ms | rows
-----+-----+-----+-----
EXPLAIN ANALYZE  | 1 | 35.78 | 0
SELECT count(*)  | 1 | 0.58 | 1
FROM transactions | 1 | 0.58 | 1
WHERE status = $1 AND amount > $2 | 1 | 0.58 | 1
SELECT pg_stat_statements_reset() | 1 | 0.58 | 1
(2 rows)
```

Bu sorgular sonrasında yukarıdaki performans izleme sorgusu çağırılmıştır. Görselde görüleceği üzere az önce index yokken 210ms süren sorgu artık 35ms'de gerçekleştirilmektedir. Böylelikle veri tabanının optimize edildiği görülmektedir.

1.3.3. Sorgu İyileştirme

Raporun bu aşamasında sorgu iyileştirmeye yönelik senaryomuz şu şekildedir:

- 2024 yılında yapılan tüm başarılı transactionların getirilmesi istenmektedir.
- transaction_date sütununda bir index mevcuttur.

Tablonun mevcut hali aşağıdaki gibidir.

```
ornek_veritabani=# \d transactions
```

Column	Type	Table "public.transactions"	Collation	Nullable	Default
id	integer			not null	nextval('transactions_id_seq'::regclass)
customer_id	integer				
transaction_date	timestamp without time zone				
amount	numeric(10,2)				
status	character varying(20)				

Indexes:

- "transactions_pkey" PRIMARY KEY, btree (id)
- "idx_transactions_date" btree (transaction_date)
- "idx_transactions_status_amount" btree (status, amount)

Görüleceği üzere işlem tarihleri transaction_date sütununda timestamp olarak saklanmaktadır. Bundan dolayı belirli bir yıldaki işlemlere erişmek için en doğru yol PostgreSQL'in EXTRACT fonksiyonunu kullanmak gibi durmaktadır.

```
ornek_veritabani=# EXPLAIN ANALYZE
SELECT sum(amount)
FROM transactions
WHERE status = 'success'
AND EXTRACT(YEAR FROM transaction_date) = 2024;
```

QUERY PLAN

```
Finalize Aggregate  (cost=147460.08..147460.09 rows=1 width=32) (actual time=248.874..249.866 rows=1.00 loops=1)
  Buffers: shared hit=12797 read=60733
  -> Gather  (cost=147459.86..147460.07 rows=2 width=32) (actual time=248.817..249.858 rows=3.00 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    Buffers: shared hit=12797 read=60733
    -> Partial Aggregate  (cost=146459.86..146459.87 rows=1 width=32) (actual time=242.317..242.318 rows=1.00 loops=3)
      Buffers: shared hit=12797 read=60733
      -> Parallel Seq Scan on transactions  (cost=0.00..146446.67 rows=5276 width=6) (actual time=0.259..229.874 rows=278181.67 loops=3)
        Filter: (((status)::text = 'success'::text) AND (EXTRACT(year FROM transaction_date) = '2024'::numeric))
        Rows Removed by Filter: 3055152
        Buffers: shared hit=12797 read=60733

Planning:
  Buffers: shared hit=31 read=3
Planning Time: 2.353 ms
Execution Time: 249.918 ms
(16 rows)
```

2024 yılında gerçekleştirilen transactionlara erişmek için yukarıdaki sorgu çalıştırıldığında transaction_date sütunu üzerine fonksiyon çağırılmaktadır. Bundan dolayı sütun üzerinde index olsa dahi indexten faydalanmak mümkün değildir.

```

Finalize Aggregate (cost=147460.08..147460.09 rows=1 width=32) (actual time=248.874..249.866 rows=1.00 loops=1)
  Buffers: shared hit=12797 read=60733
  -> Gather (cost=147459.86..147460.07 rows=2 width=32) (actual time=248.817..249.858 rows=3.00 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    Buffers: shared hit=12797 read=60733
    -> Partial Aggregate (cost=146459.86..146459.87 rows=1 width=32) (actual time=242.317..242.318 rows=1.00 loops=3)
      Buffers: shared hit=12797 read=60733
      -> Parallel Seq Scan on transactions (cost=0.00..146446.67 rows=5276 width=6) (actual time=0.259..229.874 rows=278181.67 loops=3)
        Filter: (((status)::text = 'success'::text) AND (EXTRACT(year FROM transaction_date) = '2024'::numeric))
        Rows Removed by Filter: 3055152
        Buffers: shared hit=12797 read=60733

Planning:
  Buffers: shared hit=31 read=3
Planning Time: 2.353 ms
Execution Time: 249.918 ms

```

Yukarıdaki görselde görüleceği üzere tablo üzerinde Seq Scan işlemi gerçekleştirilmiş ve indexten faydalanılamadığı için sorgu 250ms sürmüştür. Indexten faydalanabilmek için sorgumuzda index bulunan sütun üzerinde fonksiyon kullanmamak gerekmektedir. Bu doğrultuda sorgumuz aşağıdaki gibi olmalıdır.

```

SELECT sum(amount)
FROM transactions
WHERE status = 'success'
AND transaction_date >= '2024-01-01 00:00:00'
AND transaction_date < '2025-01-01 00:00:00';

```

Bu sorgu analiz edilecek olursa aşağıdaki çıktı elde edilecektir.

```

örnek_veritabanı=# EXPLAIN ANALYZE
SELECT sum(amount)
FROM transactions
WHERE status = 'success'
AND transaction_date >= '2024-01-01 00:00:00'
AND transaction_date < '2025-01-01 00:00:00';

QUERY PLAN

-----
Finalize Aggregate (cost=33780.16..33780.17 rows=1 width=32) (actual time=83.521..84.481 rows=1.00 loops=1)
  Buffers: shared hit=495137 read=4115
  -> Gather (cost=33779.94..33780.15 rows=2 width=32) (actual time=83.475..84.475 rows=3.00 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    Buffers: shared hit=495137 read=4115
    -> Partial Aggregate (cost=32779.94..32779.95 rows=1 width=32) (actual time=74.081..74.081 rows=1.00 loops=3)
      Buffers: shared hit=495137 read=4115
      -> Parallel Index Only Scan using idx_transactions_cover on transactions (cost=0.56..31895.98 rows=353585 width=6) (actual time=0.460..57.843 rows=278181.67 loops=3)
        Index Cond: ((status = 'success'::text) AND (transaction_date >= '2024-01-01 00:00:00'::timestamp without time zone) AND (transaction_date < '2025-01-01 00:00:00'::timestamp without time zone))
        Heap Fetches: 0
        Index Searches: 1
        Buffers: shared hit=495137 read=4115

Planning:
  Buffers: shared hit=24 read=1
Planning Time: 0.892 ms
Execution Time: 84.521 ms
(17 rows)

```

Yukarıdaki görselde sorgu artık 250ms değil 84ms sürmektedir.

1.4. Çıktılar

Proje kapsamında yapılan işlemlerde görüleceği üzere veri tabanı performans optimizasyonlarında sadece indeks kullanımı yeterli değildir. Veri tabanları üzerinde gerçekleştirilen sorgular uygun araçlar yardımıyla analiz edilmeli, doğru yerde doğru indeksler kullanılmalı ve sorgular bu indekslere uygun olarak hazırlanmalıdır.